

Bài Thực Hành Lab 1

I Sinh viên thực hiện:

1. Nguyễn Hoàng Thanh Tâm – 22521638
2. Nguyễn Phi Yến – 25522135

II Tính toán bit đơn giản (làm tại lớp)

Bài 1.1

Đề bài: Tính giá trị $x - y$ không dùng dấu $-$

- **Input:** Số nguyên x, y
- **Ví dụ kiểm thử (Test cases):** $\text{subtract}(10,3) == 7$ && $\text{subtract}(0,5) == -5$ && $\text{subtract}(-4,-9) == 5$

1. Ý tưởng: Để thực hiện phép trừ $x - y$ mà không dùng dấu $-$, ta áp dụng tính chất của mã bù 2: $-y == \sim y + 1$ (với $\sim$ là phép NOT đảo toàn bộ các bit). Khi đó, phép trừ $x - y$ được chuyển thành phép cộng:

$$x - y = x + (\sim y + 1)$$

2. Mã nguồn (C/C++):

```
1 int subtract (int x, int y) {  
2     return x + (~y + 1);  
3 }
```

3. Giải thích chi tiết:

- $\sim y + 1$: Lấy mã bù 2 của y (đảo toàn bộ các bit bằng toán tử $\sim$ rồi cộng 1) để tạo ra giá trị số đối $-y$.
- $x + (\dots)$: Cộng giá trị x với số đối $-y$ vừa tìm được, thu được kết quả $x + (-y) = x - y$ mà không dùng dấu trừ.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
$\text{subtract}(10,3)$	7	0x00000007
$\text{subtract}(0,5)$	-5	0xFFFFFFFF
$\text{subtract}(-4,-9)$	5	0x00000005

Bảng 1: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 1.2

Đề bài: Tính giá trị $100 \times x$ mà không sử dụng phép nhân.

- **Input:** Số nguyên x (giới hạn sao cho $100x$ biểu diễn đủ trong số nguyên có dấu 4 byte).
- **Ví dụ kiểm thử:** $\text{cal100x}(0) == 0$ && $\text{cal100x}(7) == 700$ && $\text{cal100x}(-3) == -300$

1. Ý tưởng: Sử dụng phép dịch bit sang trái ($\ll$) kết hợp phép cộng, vì $100 = 64 + 32 + 4 = 2^6 + 2^5 + 2^2$. Khi đó:

$$100x = (x \ll 6) + (x \ll 5) + (x \ll 2)$$

2. Mã nguồn (C/C++):

```
1 int cal100x(int x) {  
2     return (x << 6) + (x << 5) + (x << 2);  
3 }
```

3. Giải thích chi tiết:

- $x \ll 6$: Dịch chuyển bit sang **trái** 6 lần, tương đương với $x \times 2^6 = 64x$.
- $x \ll 5$: Dịch chuyển bit sang **trái** 5 lần, tương đương với $x \times 2^5 = 32x$.
- $x \ll 2$: Dịch chuyển bit sang **trái** 2 lần, tương đương với $x \times 2^2 = 4x$.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
$\text{cal100x}(0)$	0	0x00000000
$\text{cal100x}(7)$	700	0x000002BC
$\text{cal100x}(-3)$	-300	0xFFFFFED4

Bảng 2: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 1.3

Đề bài: Trả về nibble (nhóm 4 bit) thứ n của x (các nibble được đánh thứ tự từ 0 đến 7 từ phải sang trái).

- **Input:** Số x nguyên; Số n ($0 \leq n \leq 7$)
- **Ví dụ kiểm thử (Test cases):** $\text{getNibble}(0x12345678, 0) == 0x8$ && $\text{getNibble}(0x12345678, 5) == 0x3$ && $\text{getNibble}(0xABCDEF01, 7) == 0xA$

1. Ý tưởng: Một nibble gồm 4 bit (vị trí nibble n ứng với dịch $n \ll 2$ bit). Để lấy nibble thứ n :

- Dịch phải x sang $(n \ll 2)$ bit: $x \gg (n \ll 2)$ để đưa nibble n về 4 bit thấp nhất.
- Dùng phép $\& 0xF$ để giữ lại 4 bit này và xóa toàn bộ các bit còn lại.

2. Mã nguồn (C/C++):

```

1 int getNibble (int x, int n) {
2     int shift=n<<2;
3     return (x>>shift)&0xF;
4 }

```

3. Giải thích chi tiết: Biểu thức $(x \gg (n \ll 2)) \& 0xF$ hoạt động qua 2 bước:

- $x \gg (n \ll 2)$: Nhân n với 4 bằng phép dịch trái $n \ll 2$ để xác định số bit cần dịch. Sau đó dịch phải x để đưa nibble thứ n về vị trí 4 bit thấp nhất (bit 0 $\rightarrow$ 3).
- $\& 0xF$: Lấy kết quả AND với mask 0xF (1111 ở hệ nhị phân) để giữ nguyên giá trị 4 bit thấp nhất và xóa toàn bộ các bit phía trên về 0.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
getNibble(0x12345678,0)	0x8	0x00000008
getNibble(0x12345678,5)	0x3	0x00000003
getNibble(0xABCDEF01,7)	0xA	0x0000000A

Bảng 3: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 1.4

Đề bài: Xóa (đặt về 0) toàn bộ 8 bit của byte thứ n của số nguyên x , các byte khác giữ nguyên (các byte được đánh thứ tự từ 0 đến 3 từ phải sang trái).

- **Input:** số nguyên x , số n ($0 \leq n \leq 3$).
- **Ví dụ kiểm thử (Test cases):** $\text{clearByte}(0x12345678,1) == 0x12340078$ && $\text{clearByte}(-1,0) == 0xFFFFF000$ && $\text{clearByte}(0x5501,0) == 0x5500$

1. Ý tưởng: Dùng mask 0xFF dịch sang trái $n \times 8$ rồi Not lại để có mask thực hiện phép AND.

2. Mã nguồn (C/C++):

```

1 int clearByte(int x, int n){
2     return x & ~(0xFF<<(n<<3));
3 }

```

3. Giải thích chi tiết:

- Ví dụ $n = 2$: dịch 0xFF sang trái 16 lần ta sẽ có 0x0000FF00
Not lại $\rightarrow$ 0xFFFF00FF
AND với x để reset vị trí byte n về 0.

4. Kết quả:

Tất cả các case đều đưa ra kết quả chính xác.

Phương trình	Lý thuyết	Thực thi
clearByte(0x12345678,1)	0x12340078	0x12340078
clearByte(-1,0)	0xFFFFFFFF00	0xFFFFFFFF00
clearByte(0x5501,0)	0x5500	0x5500

Bảng 4: Kết quả sau khi test

Bài 1.5

Đề bài: Tính kết quả $x \cdot 2^n$ với n âm (tức là chia x cho 2^{-n}), không dùng phép nhân, chia.

- **Input:** Số x nguyên dương; Số n âm ($-31 \leq n \leq -1$)
- **Ví dụ kiểm thử (Test cases):** $\text{mulpw2}(20,-1) == 10 \ \&\& \ \text{mulpw2}(60,-2) == 15 \ \&\& \ \text{mulpw2}(32,-4) == 2$

1. Ý tưởng: Do n mặc định là số âm ($n < 0$), phép nhân $x \times 2^n$ tương đương với phép chia x cho 2^{-n} :

- Chuyển số âm n thành giá trị tuyệt đối $-n$ bằng quy tắc bù 2: $\sim n + 1$.
- Dịch phải x sang bên phải $(-n)$ bit bằng toán tử $\gg$.

2. Mã nguồn (C/C++):

```
1 int mulpw2 (int x, int n) {
2     int pos_n = ~n + 1;
3     return x >> pos_n;
4 }
```

3. Giải thích chi tiết:

- $\sim n + 1$: Lấy số đối của n (đổi từ số âm sang số dương tương ứng $-n$) theo dạng biểu diễn bitwise bù 2.
- $x \gg (\sim n + 1)$: Thực hiện dịch phải số x sang bên phải $(-n)$ vị trí, tương đương với phép chia x cho 2^{-n} (hay nhân x với 2^n).

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
$\text{mulpw2}(20,-1)$	10	0x0000000A
$\text{mulpw2}(60,-2)$	15	0x0000000F
$\text{mulpw2}(32,-4)$	2	0x00000002

Bảng 5: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 1.6

Đề bài: Tính phần dư $x \% 2^n$ với x là số nguyên có dấu (có thể âm); kết quả cùng dấu với x giống phép $\%$ của C.

- **Input:** Số x nguyên; Số n ($0 \leq n \leq 30$)
- **Ví dụ kiểm thử (Test cases):** $\text{remPw2}(15,2) == 3 \ \&\& \ \text{remPw2}(-15,2) == -3$
 $\&\& \ \text{remPw2}(-16,3) == 0 \ \&\& \ \text{remPw2}(-1,4) == -1$
- **Gợi ý:** Tính thương $q = x/2^n$ làm tròn về 0 bằng cách cộng độ lệch (bias) khi x âm (CSAPP mục 2.3.7), rồi lấy $x - q \cdot 2^n$.

1. Ý tưởng: Tính thương $q = x/2^n$ làm tròn về 0 bằng cách cộng độ lệch (bias) khi x âm (CSAPP mục 2.3.7), rồi lấy $x - q \cdot 2^n$.

2. Mã nguồn (C/C++):

```
1 int remPw2(int x, int n){
2     int sign = x >> 31;
3     int bias = (1 << n) + ~0;
4     int q = (x + (bias & sign)) >> n;
5     return x + ~(q << n) + 1;
6 }
```

3. Giải thích chi tiết:

- Thực hiện dịch phải số nguyên có dấu 31 bit để lấy bit dấu của x
- Tính giá trị hiệu chỉnh (bias) cho số âm
- Tính thương số nguyên $q = x/2^n$ làm tròn về phía 0
- Tính phần dư theo công thức: Số dư $= x - (q \times 2^n)$.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
$\text{remPw2}(15,2)$	3	0x00000003
$\text{remPw2}(-15,2)$	-3	0xFFFFFFFFD
$\text{remPw2}(-16,3)$	0	0x00000000
$\text{remPw2}(-1,4)$	-1	0xFFFFFFFF

Bảng 6: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

III Các phép kiểm tra dựa trên tính toán bit (làm tại lớp)

Bài 2.1

Đề bài: Kiểm tra y có phải là số đối của x hay không ($y = -x$ theo toán học). Trả về 1 nếu đúng, 0 nếu sai. Lưu ý trường hợp số 0 (số đối của 0 là 0).

- **Input:** Số nguyên x, y

- **Ví dụ kiểm thử (Test cases):** `isNegation(4,-4) == 1 && isNegation(0,0) == 1 && isNegation(5,-6) == 0 && isNegation(3,3) == 0`

1. Ý tưởng: Đề y là số đối của x ($y = -x$):

- Tổng $x + y$ phải bằng 0 (kiểm tra bằng `!(x + y)`).
- Loại trừ trường hợp đặc biệt $x = y = T_{min}$ (`0x80000000`) vì $-T_{min} = T_{min}$ trong số bù 2.
- Do T_{min} là số âm duy nhất thỏa mãn $x + y = 0$, ta chỉ cần loại bỏ các trường hợp $x < 0$ bằng cách kiểm tra bit dấu `x >> 31`.

2. Mã nguồn (C/C++):

```
1 int isNegation(int x, int y) {
2     return !(x + y) & !((x & y) >> 31);
3 }
```

3. Giải thích chi tiết:

- `!(x + y)`: Kiểm tra tổng $x + y = 0$. Trả về 1 nếu tổng bằng 0, ngược lại trả về 0.
- `x >> 31`: Dịch phải số học 31 bit để lấy bit dấu. Trả về `0xFFFFFFFF` (khác 0) nếu $x < 0$ và trả về 0 nếu $x \geq 0$.
- `!(x >> 31)`: Phủ định logic bit dấu. Trả về 0 nếu $x < 0$ (loại T_{min}) và trả về 1 nếu $x \geq 0$.
- `!(x + y) & !(x >> 31)`: Kết hợp 2 điều kiện bằng toán tử `&` để đảm bảo tổng bằng 0 và x là số không âm.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
<code>isNegation(4,-4)</code>	1	<code>0x00000001</code>
<code>isNegation(0,0)</code>	1	<code>0x00000001</code>
<code>isNegation(5,-6)</code>	0	<code>0x00000000</code>
<code>isNegation(3,3)</code>	0	<code>0x00000000</code>
<code>isNegation(INT_MIN,INT_MIN)</code>	0	<code>0x00000000</code>

Bảng 7: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 2.2

Đề bài: Kiểm tra số nguyên x có chia hết cho 8 nhưng không chia hết cho 16 hay không. Trả về 1 nếu đúng, 0 nếu sai.

- **Input:** Số nguyên x không âm
- **Ví dụ kiểm thử:** `isMul8Not16(8) == 1 && isMul8Not16(24) == 1 && isMul8Not16(16) == 0 && isMul8Not16(12) == 0`

1. Ý tưởng: Vì số bit từ vị trí 2^4 trở đi (32, 64,...) đều là bội số của 16 nên ta sẽ xét xem nó có chia hết cho 8 không, sau đó sẽ xét các bit có trọng số từ 2^4 trở đi xem liệu có bit 1 hay không.

2. Mã nguồn (C/C++):

```
1 int isMul8Not16(int x){
2     return !(x&7) & !(x&8);
3 }
```

3. Giải thích chi tiết:

- Đầu tiên ta dùng hàm $!(x \& 7)$ để so xem có chia hết cho 8 hay không vì 8 không thể chia hết cho số từ 1 $\rightarrow$ 7:

```
1 000 ... 0111 (7)
2 000 ... 1000 (8)
3 -----
4 000 ... 0000 (0)
```

- Sau đó ta dùng hàm $!(x \& 8)$ để xem có chia hết cho 16 hay không:

```
1 000 ... 11000 (24)
2 000 ... 01000 (8)
3 -----
4 000 ... 01000 (8)
```

$8 \rightarrow \text{True}$: suy ra khi dùng ! sẽ thành False, dùng ! một lần nữa để đảo ngược thành True.

- Ví dụ trường hợp cho $x = 32$:

```
1 000 ... 000111 (7)      000 ... 001000 (8)
2 000 ... 100000 (32)     000 ... 100000 (32)
3 -----                -----
4 000 ... 000000 (0)      000 ... 000000 (0)
```

Như vậy thì $!(x \& 7) = \text{True}$ và $!(x \& 8) = \text{True} \rightarrow$ Số chia được cho 8 mà không chia được cho 16

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
isMul8Not16(8)	1	0x00000001
isMul8Not16(24)	1	0x00000001
isMul8Not16(16)	1	0x00000001
isMul8Not16(12)	0	0x00000000

Bảng 8: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 2.3

Đề bài: Trả về 1 nếu x không dương ($x \leq 0$)

- **Input:** Số nguyên x

- **Ví dụ kiểm thử (Test cases):** `isNonPositive(10) == 0 && isNonPositive(-5) == 1 && isNonPositive(0) == 1`

1. Ý tưởng: Số x không dương ($x \leq 0$) khi và chỉ khi x là số âm ($x < 0$) hoặc $x = 0$:

- Kiểm tra $x < 0$: Lấy bit đầu của x bằng cách dịch phải 31 bit (`x >> 31`) & 1. Kết quả trả về 1 nếu x âm và 0 nếu $x \geq 0$.
- Kiểm tra $x = 0$: Dùng phủ định logic `!x`. Trả về 1 nếu $x = 0$ và 0 nếu $x \neq 0$.
- Kết hợp 2 điều kiện bằng toán tử OR bitwise `|` để trả về 1 nếu $x \leq 0$.

2. Mã nguồn (C/C++):

```
1 int isNonPositive(int x) {
2     int sign=(x>>31)&1;
3     int is_zero=!x;
4     return sign|is_zero;
5 }
```

3. Giải thích chi tiết:

- `(x >> 31) & 1`: Dịch bit đầu về vị trí 0 và dùng mask & 1 để trích xuất bit đầu (trả về 1 khi $x < 0$, trả về 0 khi $x \geq 0$).
- `!x`: Phép phủ định logic trả về 1 khi $x = 0$ và trả về 0 khi $x \neq 0$.
- `((x >> 31) & 1) | !x`: Thực hiện phép OR giữa hai điều kiện. Kết quả bằng 1 nếu x thỏa mãn là số âm hoặc là số 0, ngược lại bằng 0 đối với số dương ($x > 0$).

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
<code>isNonPositive(10)</code>	0	0x00000000
<code>isNonPositive(-5)</code>	1	0x00000001
<code>isNonPositive(0)</code>	1	0x00000001

Bảng 9: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 2.4

Đề bài: Trả về 1 nếu bit 1 cao nhất của x nằm đúng ở vị trí n , tức $2^n \leq x < 2^{n+1}$.

- **Input:** Số x nguyên dương; Số n ($0 \leq n \leq 30$)
- **Ví dụ kiểm thử (Test cases):** `isTopBit(5,2) == 1 && isTopBit(8,2) == 0 && isTopBit(7,3) == 0 && isTopBit(1,0) == 1`

1. Ý tưởng: Ta lùi n bit để xem liệu từ bit $n+1$ còn bit nào bằng 1 không.

2. Mã nguồn (C/C++):

```

1 int isTopBit(int x, int n){
2     return !((x>>n) ^1);
3 }

```

3. Giải thích chi tiết:

- Giả sử x có bit 1 cao nhất ở n . Ta dịch phải n bit $\rightarrow$ bit thứ n giờ ở vị trí LSB. XOR với 1 để xem liệu các bit trước vị trí LSB có 1 hay không. Nếu có xuất ra 0 nhờ dấu !

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
isTopBit(5,2)	1	0x00000001
isTopBit(8,2)	0	0x00000000
isTopBit(7,3)	0	0x00000000
isTopBit(1,0)	1	0x00000001

Bảng 10: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 2.5

Đề bài: Trả về 1 nếu phép tính $x + y$ không bị tràn số (overflow) trên số nguyên 32 bit có dấu, ngược lại trả về 0.

- Input:** Số nguyên x, y
- Ví dụ kiểm thử (Test cases):** $\text{addOK}(3,4) == 1 \ \&\& \ \text{addOK}(0x7FFFFFFF,1) == 0 \ \&\& \ \text{addOK}(0x80000000,-1) == 0 \ \&\& \ \text{addOK}(0x80000000,0x7FFFFFFF) == 1 \ \&\& \ \text{addOK}(-1,-1) == 1$

1. Ý tưởng: Hiện tượng tràn số (overflow) trong phép cộng hai số nguyên có dấu chỉ xảy ra khi hai số x và y cùng dấu, nhưng tổng $(x + y)$ lại mang dấu ngược lại:

- Lấy bit dấu của x, y và $\text{sum} = x + y$.
- Hai số x và y cùng dấu khi $\text{sign}_x \wedge \text{sign}_y$ bằng 0, hay $!(\text{sign}_x \wedge \text{sign}_y)$ bằng 1.
- Tổng sum khác dấu với x khi $\text{sign}_x \wedge \text{sign}_{\text{sum}}$ bằng 1.
- Hiện tượng tràn số xảy ra khi cả hai điều kiện trên đồng thời đúng. Do đó, phép cộng hợp lệ (không tràn) khi phủ định logic ! của điều kiện tràn số bằng 1.

2. Mã nguồn (C/C++):

```

1 int addOK (int x, int y) {
2     int sum=x+y;
3     int sign_x=(x>>31)&1;
4     int sign_y=(y>>31)&1;
5     int sign_sum=(sum>>31)&1;
6     return !((~(sign_x ^ sign_y)) & (sign_x ^ sign_sum));
7 }

```

3. Giải thích chi tiết:

- `int sum = x + y;`: Tính tổng hai số x và y .
- `sign_x, sign_y, sign_sum`: Lấy bit dấu của x, y và sum (bằng 1 nếu âm, 0 nếu không âm) bằng cách dịch phải 31 bit và nhân logic với 1.
- `!(sign_x ^ sign_y)`: Phép XOR trả về 0 nếu $sign_x = sign_y$ (hai số cùng dấu). Phủ định logic ! biến 0 thành 1, đánh dấu điều kiện "cùng dấu" thỏa mãn.
- `sign_x ^ sign_sum`: Phép XOR trả về 1 nếu bit dấu của tổng sum khác với bit dấu ban đầu của x (kết quả bị đổi dấu).
- `!( ... & ... )`: Nhân logic hai điều kiện trên để kiểm tra xem có tràn số hay không, sau đó dùng phủ định logic ! để trả về 1 nếu không bị tràn số (hợp lệ) và 0 nếu bị tràn số.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
<code>addOK(3,4)</code>	1	0x00000001
<code>addOK(0x7FFFFFFF,1)</code>	0	0x00000000
<code>addOK(0x80000000,-1)</code>	0	0x00000000
<code>addOK(0x80000000,0x7FFFFFFF)</code>	1	0x00000001
<code>addOK(-1,-1)</code>	1	0x00000001

Bảng 11: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 2.6

Đề bài: Trả về 1 nếu tất cả các bit ở vị trí lẻ (bit 1, 3, 5, ..., 31) của x đều bằng 1, ngược lại trả về 0.

- **Input:** Số nguyên x
- **Ví dụ kiểm thử (Test cases):** `allOddBits(0xFFFFFFFF) == 0` &&
`allOddBits(0xAAAAAAAA) == 1` && `allOddBits(0xFFFFFFFF) == 1` &&
`allOddBits(0x2AAAAAAAA) == 0`
- **Gợi ý:** hằng số tối đa là 0xFF, cần tự tạo mask 0xAAAAAAAA từ 0xAA; hoặc gấp đôi x bằng phép `<<` (16, 8 bit).

1. Ý tưởng:

2. Mã nguồn (C/C++):

```
1 int allOddBits(int x){
2     int mask = 0xAA;
3     mask = mask | (mask<<8);
4     mask = mask | (mask<<16);
5     return !((x & mask) ^ mask);
6 }
```

3. Giải thích chi tiết:

- Tạo mặt nạ 8 bit cơ sở với giá trị 0xAA (mã nhị phân: 10101010), trong đó các bit ở vị trí lẻ (1, 3, 5, 7) đều bằng 1.
- Mở rộng mặt nạ lên 16 bit bằng cách dịch trái 8 bit rồi thực hiện phép OR.
- Mở rộng tiếp mặt nạ lên đủ 32 bit
- Lọc lấy các bit ở vị trí lẻ của x và xóa toàn bộ các bit ở vị trí chẵn về 0. So sánh kết quả vừa lọc với mask.
- Nếu tất cả các bit vị trí lẻ của x đều bằng 1, thì (x & mask) sẽ đúng bằng mask. Khi đó mask XOR mask = 0, còn không sẽ bằng 1.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
allOddBits(0xFFFFFFFFD)	0	0x00000000
allOddBits(0xAAAAAAAAA)	1	0x00000001
allOddBits(0xFFFFFFFFF)	1	0x00000001
allOddBits(0x2AAAAAAAAA)	0	0x00000000

Bảng 12: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

IV Số thực dấu phẩy động (làm tại nhà)

Bài 3.1

Đề bài: Trả về mẫu bit của $-|f|$. Nếu f là NaN thì trả về nguyên uf.

- **Input:** Mẫu bit uf của số float f
- **Ví dụ kiểm thử (Test cases):** float_sign(0x3F800000) == 0xBF800000 && float_sign(0xC0200000) == -1 && float_sign(0x80000000) == 0 && float_sign(0x7FC00000) == 0 && float_sign(0x00000001) == 1

1. **Ý tưởng:** Ép bit dấu thành 1 và giữ nguyên 31 bit còn lại.

2. **Mã nguồn (C/C++):**

```
1 unsigned float_nabs(unsigned uf) {  
2     if ((uf & 0x7FFFFFFF) > 0x7F800000) {  
3         return uf;  
4     }  
5     return uf | 0x80000000;  
6 }
```

3. Giải thích chi tiết:

- Xét xem uf có vượt qua NaN không, nếu có thì giữ nguyên uf
- Phép toán uf | 0x80000000 sẽ bật bit dấu (bit thứ 31) lên 1 và giữ nguyên tất cả các bit còn lại, đảm bảo trả về đúng giá trị.

4. Kết quả:

Tất cả các case đều đưa ra kết quả chính xác.

Phương trình	Lý thuyết	Thực thi
<code>float_sign(0x3F800000)</code>	<code>0xBF800000</code>	<code>0xBF800000</code>
<code>float_sign(0xC0200000)</code>	<code>0xC0200000</code>	<code>0xC0200000</code>
<code>float_sign(0x80000000)</code>	<code>0x80000000</code>	<code>0x80000000</code>
<code>float_sign(0x7FC00000)</code>	<code>0x7FC00000</code>	<code>0x7FC00000</code>

Bảng 13: Kết quả sau khi test

Bài 3.2

Đề bài: Trả về 1 nếu $f > 0$, -1 nếu $f < 0$, và 0 nếu f là +0, -0 hoặc NaN.

- **Input:** Mẫu bit `uf` của số float f
- **Ví dụ kiểm thử (Test cases):** `float_sign(0x3F800000) == 1 && float_sign(0xC0200000) == -1 && float_sign(0x80000000) == 0 && float_sign(0x7FC00000) == 0 && float_sign(0x00000000) == 1`

1. Ý tưởng: Dựa vào quy chuẩn số thực dấu phẩy động IEEE 754 32-bit:

- Tách mẫu bit `uf` thành 3 thành phần: bit dấu `sign` (bit 31), phần mũ `exp` (8 bit từ 22 đến 30) và phần phân số `frac` (23 bit từ 0 đến 22).
- **Trường hợp trả về 0:**
 - **Số 0 (+0 hoặc -0):** Bỏ bit dấu (xét 31 bit còn lại `uf & 0x7FFFFFFF`), nếu bằng 0 thì giá trị là +0 hoặc -0.
 - **Số NaN:** Xảy ra khi 8 bit phần mũ đều bằng 1 (`exp == 0xFF`) và 23 bit phần phân số khác 0 (`frac != 0`).
- **Trường hợp trả về -1:** Khi không phải số 0 hay NaN, và bit dấu `sign == 1` (số thực âm $f < 0$).
- **Trường hợp trả về 1:** Khi không phải số 0 hay NaN, và bit dấu `sign == 0` (số thực dương $f > 0$).

2. Mã nguồn (C/C++):

```

1 int float_sign(unsigned uf) {
2     unsigned exp = (uf >> 23) & 0xFF;
3     unsigned frac = uf & 0x7FFFFFFF;
4     unsigned isZero = !(uf & 0x7FFFFFFF);
5     unsigned isNaN = !(exp ^ 0xFF) & !!frac;
6     unsigned sign = (uf >> 31) & 1;
7     return (isZero | isNaN) ? 0 : (sign ? -1 : 1);
8 }

```

3. Giải thích chi tiết:

- `unsigned exp = (uf >> 23) & 0xFF;`: Trích xuất 8 bit phần mũ của mẫu bit `uf`.
- `unsigned frac = uf & 0x7FFFFFFF;`: Trích xuất 23 bit phần phân số.

- `unsigned isZero = !(uf & 0x7FFFFFFF);`: Phép AND giữ lại 31 bit dữ liệu (xóa bit dấu). Phủ định logic `!` trả về 1 nếu cả 31 bit bằng 0 (+0 hoặc -0).
- `unsigned isNaN = !(exp ^ 0xFF) & !!frac;: !(exp ^ 0xFF)` trả về 1 khi `exp == 0xFF`. `!!frac` biến `frac` khác 0 thành 1. Phép AND trả về 1 nếu là NaN.
- `unsigned sign = (uf >> 31) & 1;`: Trích xuất bit dấu ở vị trí 31 (1 là âm, 0 là dương).
- `(isZero | isNaN) ? 0 : (sign ? -1 : 1)`: Nếu rơi vào trường hợp số 0 hoặc NaN thì trả về 0. Ngược lại, dựa vào bit dấu để trả về -1 đối với $f < 0$ hoặc 1 đối với $f > 0$.

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
<code>float_sign(0x3F800000)</code>	1	0x00000001
<code>float_sign(0xC0200000)</code>	-1	0xFFFFFFFF
<code>float_sign(0x80000000)</code>	0	0x00000000
<code>float_sign(0x7FC00000)</code>	0	0x00000000
<code>float_sign(0x00000001)</code>	1	0x00000001

Bảng 14: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 3.3

Đề bài: Trả về mẫu bit của số float 2.0^x . Nếu kết quả nhỏ hơn số không chuẩn hóa nhỏ nhất thì trả về 0; nếu quá lớn thì trả về $+\infty$.

- **Input:** Số nguyên x
- **Ví dụ kiểm thử (Test cases):** `float_pwr2(0) == 0x3F800000 && float_pwr2(-1) == 0x3F000000 && float_pwr2(127) == 0x00400000` (số không chuẩn hóa)
`&& float_pwr2(-149) == 0x00000001 && float_pwr2(-150) == 0 && float_pwr2(128) == 0x7F800000 (+∞)`
- **Gợi ý:** chia x thành 4 khoảng: quá nhỏ, không chuẩn hóa, chuẩn hóa, quá lớn (CSAPP bài tập 2.90).

1. Ý tưởng: Tính 2^x dưới dạng bit IEEE-754 thực chất chỉ là thao tác đặt bit 1 vào đúng vị trí trên thanh ghi (vùng Mũ E hoặc vùng Lẻ M), hoàn toàn không cần thực hiện phép nhân hay chia số thực nào.

2. Mã nguồn (C/C++):

```

1 unsigned float_pwr2(int x) {
2     if (x < -149) {
3         return 0;
4     }
5     if (x <= -127) {
6         return 1 << (x + 149);
7     }
8     if (x <= 127) {

```

```

9         return (x + 127) << 23;
10     }
11     return 0x7F800000;
12 }

```

3. Giải thích chi tiết:

- $x < -149$ (Quá nhỏ): Số 2^x nhỏ hơn cả số không chuẩn hóa nhỏ nhất (2^{-149}) nên không thể biểu diễn $\rightarrow$ Trả về 0.
- $-149 \leq x \leq -127$ (Số không chuẩn hóa): Trường mũ $E = 0$, ta bật duy nhất 1 bit ở trường lẻ M tại vị trí tương ứng $\rightarrow 1 \ll (x + 149)$.
- $-126 \leq x \leq 127$ (Số chuẩn hóa): Trường lẻ $M = 0$, ta tính phần mũ thực tế bằng cách cộng độ lệch bias 127 rồi dịch sang vùng mũ E (từ bit thứ 23) $\rightarrow (x + 127) \ll 23$.
- $x \geq 128$ (Quá lớn): Bị tràn số dương do phần mũ $E \geq 255 \rightarrow$ Trả về $+\infty$ (0x7F800000).

4. Kết quả:

Phương trình	Lý thuyết	Thực thi
float_pwr2(0)	0x3F800000	0x3F800000
float_pwr2(-1)	0x3F000000	0x3F000000
float_pwr2(127)	0x00400000 (số không chuẩn hóa)	0x7F000000
float_pwr2(-149)	0x00000001	0x00000001
float_pwr2(-150)	0	0x00000000
float_pwr2(128)	0x7F800000 ($+\infty$)	0x7F800000

Bảng 15: Kết quả sau khi test

Tất cả các case đều đưa ra kết quả chính xác.

Bài 3.4

Đề bài: Trả về 1 nếu f là một số nguyên (kể cả $+0$ và -0); trả về 0 nếu f có phần thập phân khác 0, hoặc f là NaN, $\pm\infty$

- **Input:** Mẫu bit uf của số float f
- **Ví dụ kiểm thử (Test cases):** $\text{float_isInt}(0x40400000) == 1$ (3.0) && $\text{float_isInt}(0x3FC00000) == 0$ (1.5) && $\text{float_isInt}(0x80000000) == 1$ (-0) && $\text{float_isInt}(0x3F000000) == 0$ (0.5) && $\text{float_isInt}(0x4B000001) == 1$ (8388609.0) && $\text{float_isInt}(0x7F800000) == 0$ ($+\infty$)

1. Ý tưởng: Dựa vào chuẩn IEEE 754 cho số thực 32-bit, giá trị phần mũ thực tế là $E = \text{exp} - 127$:

- **Trường hợp $\text{exp} = 255$:** Đại diện cho NaN hoặc $\pm\infty$, trả về 0.
- **Trường hợp $E < 0$ ($\text{exp} < 127$):** Giá trị $|f| < 1$. Chỉ có hai số đặc biệt $+0$ và -0 (khi 31 bit còn lại bằng 0) là số nguyên (trả về 1), các số khác đều có phần thập phân khác 0 (trả về 0).

- **Trường hợp $E \geq 23$:** Dấu phẩy động dịch sang phải từ 23 vị trí trở lên, bao trùm toàn bộ 23 bit của frac, do đó phần lẻ bằng 0 nên luôn là số nguyên (trả về 1).
- **Trường hợp $0 \leq E < 23$:** Dấu phẩy động nằm trong khoảng 23 bit của frac. Số là số nguyên khi và chỉ khi $23 - E$ bit thấp nhất của frac đều bằng 0.

2. Mã nguồn (C/C++):

```

1 int float_isInt(unsigned uf) {
2     unsigned exp = (uf >> 23) & 0xFF;
3     unsigned frac = uf & 0x7FFFFFFF;
4     int E = exp - 127;
5     if (exp == 0xFF) {
6         return 0;
7     }
8     if (E < 0) {
9         if ((uf & 0x7FFFFFFF) == 0) {
10            return 1;
11        }
12        return 0;
13    }
14    if (E >= 23) {
15        return 1;
16    }
17    if ((frac & ((1 << (23 - E)) - 1)) != 0) {
18        return 0;
19    }
20    return 1;
21 }

```

3. Giải thích chi tiết:

- `unsigned exp = (uf >> 23) & 0xFF;`: Trích xuất 8 bit phần mũ.
- `unsigned frac = uf & 0x7FFFFFFF;`: Trích xuất 23 bit phần phân số.
- `int E = exp - 127;`: Tính số mũ thực tế E .
- `if (exp == 0xFF) return 0;`: Loại bỏ số NaN và $\pm\infty$.
- `if (E < 0)`: Xét $0 < |f| < 1$. Nếu `(uf & 0x7FFFFFFF) == 0` thì đó là $+0$ hoặc -0 nên trả về 1, ngược lại có phần thập phân nên trả về 0.
- `if (E >= 23) return 1;`: Dấu phẩy động dịch qua hết 23 bit của phần phân số, không còn phần thập phân, trả về 1.
- `(1 << (23 - E)) - 1`: Tạo mask kiểm tra $23 - E$ bit dư ở phần thập phân.
- `if ((frac & ((1 << (23 - E)) - 1)) != 0) return 0;`: Phép AND với mask kiểm tra xem có bit 1 nào nằm ở phần thập phân không. Nếu có trả về 0, ngược lại trả về 1.

4. Kết quả:

Tất cả các case đều đưa ra kết quả chính xác.

Phương trình	Lý thuyết	Thực thi
float_isInt(0x40400000)	1	0x00000001
float_isInt(0x3FC00000)	0	0x00000000
float_isInt(0x80000000)	1	0x00000001
float_isInt(0x3F000000)	0	0x00000000
float_isInt(0x4B000001)	1	0x00000001
float_isInt(0x7F800000)	0	0x00000000

Bảng 16: Kết quả sau khi test

V Tổng kết

BẢNG TỔNG KẾT KẾT QUẢ BÀI TẬP

Để kiểm chứng những đoạn code trên có lỗi hổng hay không, chúng em đã tạo thêm một file **check_datalab.py** đã được push trên GitHub ở thư mục [Lab1-Nhom7-25521638-2552135](#). Kết quả kiểm tra như sau:

```
=====
BẢNG TỔNG KẾT
=====
```

STT	Hàm	Test đúng	Kết quả	Ops	Max	So Max	Luật	Điểm	Kết luận	Ghi chú
1.1	subtract	587/587	Đúng hết	3	8	OK	OK	0.5/0.5	ĐẠT	
1.2	cal100x	145/145	Đúng hết	5	10	OK	OK	1/1	ĐẠT	
1.3	getNibble	399/399	Đúng hết	3	10	OK	OK	1/1	ĐẠT	
1.4	clearByte	275/275	Đúng hết	4	20	OK	OK	1.5/1.5	ĐẠT	
1.5	mulpw2	1052/1052	Đúng hết	3	8	OK	OK	1.5/1.5	ĐẠT	
1.6	remPw2	1113/1113	Đúng hết	11	20	OK	OK	1.5/1.5	ĐẠT	
2.1	isNegation	510/510	Đúng hết	4	8	OK	OK	1.5/1.5	ĐẠT	
2.2	isMul8Not16	156/156	Đúng hết	6	10	OK	OK	0.5/0.5	ĐẠT	
2.3	isNonPositive	169/169	Đúng hết	4	15	OK	OK	1/1	ĐẠT	
2.4	isTopBit	1152/1152	Đúng hết	3	20	OK	OK	1/1	ĐẠT	
2.5	addOK	1186/1186	Đúng hết	12	20	OK	OK	1.5/1.5	ĐẠT	
2.6	allOddBits	214/214	Đúng hết	7	20	OK	OK	1.5/1.5	ĐẠT	
3.1	float_nabs	187/187	Đúng hết	3	10	OK	OK	1/1	ĐẠT	
3.2	float_sign	187/187	Đúng hết	14	15	OK	OK	1/1	ĐẠT	
3.3	float_pwr2	151/151	Đúng hết	9	30	OK	OK	2/2	ĐẠT	
3.4	float_isInt	337/337	Đúng hết	14	30	OK	OK	2/2	ĐẠT	

```
=====
Số câu ĐẠT (đúng hết test + ops <= Max Ops + không vi phạm luật): 16/16
Điểm ước tính: 20 / 20 (sai hoặc vi phạm luật = 0; đúng nhưng vượt Max Ops = chia đôi)
Lưu ý: số ops do script tự đếm (heuristic); công cụ chấm chính thức của GV có thể đếm hơi khác.
PS D:\LTHT> █
```

Hình 1: Kết quả sau khi chạy test case